Student Placement Prediction Using Machine Learning: A Comparative Study of Classification and Clustering on Indian Engineering Students

Solanki Parth ,Tanna Swayam, Thakkar Jay
Department of Computer Science and Engineering
Unitedworld Institute of Technology, Karnavati University
Gandhinagar, India
KUIDs: ku2407u649, ku2407u650, ku2407u651
{ku2407u649, ku2407u650, ku2407u651}@karnavatiuniversity.edu.in

***Abstract*—Getting a campus placement is something that every engineering student works towards for four years. But identifying which students are likely to get placed — and which ones need more support — is still done manually at most colleges, which is slow and often inconsistent. This paper explores whether machine learning can do a better job of it. We worked with a dataset of 5,000 Indian engineering students from five different branches, covering 22 features including CGPA, backlogs, internships, skill ratings, and even socio-economic background. Two supervised classification models — K-Nearest Neighbours (KNN) and Decision Tree — were trained to directly predict placement outcomes. On top of that, we also applied K-Means clustering to see whether the data naturally separates into placed and not-placed groups without being told the labels. After preprocessing that included missing value handling, one-hot encoding, Min-Max scaling, and an 80:20 train-test split, KNN achieved an accuracy of 86.00% and Decision Tree reached 85.10%. K-Means, when its clusters were mapped to placement labels, achieved a surprisingly competitive 86.06%. The results confirm that CGPA and backlogs are the strongest drivers of placement outcomes, and that both classification and unsupervised clustering can be useful tools for colleges trying to build smarter placement support systems.**

***Index Terms*—student placement prediction, K-nearest neighbours, decision tree, K-means clustering, machine learning, CGPA, educational data mining, feature importance, Indian engineering students**

I. Introduction

Ask any Indian engineering student what they are most worried about in their final year, and the answer is almost always the same — placement. It is not just a milestone; for a lot of students and their families, it is the entire reason they chose engineering in the first place. And yet, every placement season, a significant number of students miss out, often without any early warning that they were at risk.

The challenge for Training and Placement Officers (TPOs) is real. They are dealing with hundreds — sometimes thousands — of students, very limited time, and no systematic way to figure out who needs extra help before the placement drive begins. The traditional approach of looking at marks and manually shortlisting students just does not scale well.

Machine learning gives us a much better option. By training a model on historical student data, we can build a system that looks at a student's academic record, skill ratings, internship history, and other details, and predicts whether they are likely to get placed. Even more interesting is the question of whether the data itself already forms natural groups — placed and not placed — without us having to tell it. That is exactly the kind of thing clustering algorithms like K-Means are designed to explore.

In this paper, we apply and compare three machine learning models on a real dataset of 5,000 Indian engineering students:

1) **K-Nearest Neighbours (KNN)** — a classification model that predicts a student's placement outcome by looking at the k most similar students in the training data.
2) **Decision Tree (DT)** — a classification model that learns a set of if-else rules from the training data to separate placed and not-placed students.
3) **K-Means Clustering** — an unsupervised model that groups students into two clusters based purely on their feature similarity, without using the placement label during training.

By combining supervised classification with unsupervised clustering, this study offers a more complete picture of how machine learning can be applied to the placement prediction problem from two very different angles.

The paper is structured as follows. Section II reviews related work. Section III describes the dataset. Section IV covers data preprocessing. Section V explains the methodology. Section VI describes each model mathematically. Section VII defines the evaluation metrics. Section VIII presents and discusses the results. Section IX concludes with future directions.

II. Related Work

Predicting student placement using machine learning has been an active area of research for several years now. Earlier studies typically used small, single-institution datasets with a handful of features — mostly CGPA and communication scores — and applied algorithms like Naive Bayes, SVM, and basic decision trees [4]. These gave decent results in controlled settings but were never really tested at scale.

More recent work has moved towards ensemble approaches. Random Forest and Gradient Boosting models [7] have shown consistent improvements over individual classifiers because they reduce variance by combining many trees. Studies focusing on Indian engineering colleges tend to highlight academic performance metrics as the most important predictors, which is consistent with how most Indian companies structure their campus recruitment process.

What most of these studies still lack, however, is a broader and more diverse feature set. Restricting the input to just marks and grades misses a lot of important signals — like whether a student has done internships, how many projects they have completed, their stress levels, or even their city background. Our dataset addresses this by including 22 features across academic, skill, co-curricular, and socio-economic dimensions.

Another gap in prior work is the absence of unsupervised approaches. Almost all placement prediction studies treat this purely as a supervised classification task. We take a different angle by also applying K-Means clustering to see whether the dataset's natural structure aligns with the placement labels — which adds an exploratory dimension that classification alone cannot offer [1].

III. Dataset Description

A. Overview

The dataset used in this study combines two separate CSV files joined on the common key `Student_ID`. The primary file contains student profile features, and the secondary file contains placement labels and salary information. After merging, the final dataset has 5,000 records and 25 columns. Table **??** summarises the key statistics.

TABLE I
Model Performance on Test Set (N = 1,000)

Model	Accuracy	Precision	Recall	F1-Score
KNN ($k = 5$)	86.50%	0.874	0.985	0.926
Decision Tree	84.60%	0.903	0.920	0.911
K-Means ($k = 2$)	86.06%	N/A	N/A	N/A

B. Feature Categories

The 22 features fall into four broad groups:

(1) Academic Features: CGPA (range 5.0–10.0), 10th percentage, 12th percentage, number of active backlogs, daily study hours, and attendance percentage. These are the most traditional indicators and, as we will see in the results, the most influential ones.

(2) Skill Ratings: Coding skill (1–5), communication skill (1–5), and aptitude rating (1–5). These capture how technically and interpersonally prepared a student is for the job market.

(3) Co-curricular Indicators: Number of projects completed, internships done, hackathons attended, certifications earned, and extracurricular involvement (Low / Medium / High).

(4) Socio-economic and Lifestyle Factors: Family income level, city tier (1/2/3), internet access, part-time job status, average sleep hours, and stress level (1–10). These features are often left out in similar studies, but they can have a real impact on how prepared a student is when placement season comes around.

C. Target Variable

The placement outcome is a binary label: *Placed* (encoded as 1) or *Not Placed* (encoded as 0). As shown in Fig. 5, the dataset is heavily skewed towards placed students (86.06%), which means recall becomes especially important — we want to make sure that the small group of not-placed students is not overlooked by the model.

IV. Data Preprocessing

Raw data rarely comes ready for a machine learning model. We applied the following four preprocessing steps in sequence.

Step 1 — Dataset Merging and Column Removal. Both CSV files were merged using an inner join on `Student_ID`. After merging, `Student_ID` was dropped since it is just an identifier. The column `salary_lpa` was also removed because it is only available for students who were actually placed — keeping it would create data leakage and give the model an unfair advantage during training.

Step 2 — Handling Missing Values. The `extracurricular_involvement` column had 1,006 missing entries (about 20% of the data). Rather than dropping all these rows and losing a large chunk of the dataset, we filled them with the most frequent value (*Medium*) using mode imputation. This is a standard and reasonable approach for ordinal categorical features.

Step 3 — Encoding Categorical Features. Categorical columns such as `gender`, `branch`, `city_tier`, `family_income_level`, `stress_level`, `extracurricular_involvement`, and others were converted to numeric form using one-hot encoding with `drop_first=True`, which avoids multicollinearity from redundant dummy variables.

Step 4 — Feature Scaling. All numeric features were scaled to the $[0, 1]$ range using Min-Max normalisation:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

This step is particularly critical for KNN and K-Means, both of which compute distances between data points. Without scaling, a high-magnitude feature like `tenth_percentage` (range 0–100) would completely overshadow a low-magnitude feature like `backlogs` (range 0–5) in the distance calculation.

Step 5 — Train-Test Split. The final preprocessed dataset was divided into 80% training (4,000 samples) and 20% testing (1,000 samples) using a stratified split with `random_state=42` to ensure reproducibility and maintain the same class ratio in both sets.

V. Proposed Methodology

A. System Overview

The overall pipeline for this study is as follows:

Load CSVs → Merge → Preprocess → Split → Train Models → Evaluate

All three models are trained independently on the same data. KNN and Decision Tree are supervised classifiers — they learn from labelled examples and predict placement outcomes on unseen students. K-Means, on the other hand, is an unsupervised clustering algorithm — it groups students by feature similarity alone, without ever seeing the placement labels during training. After clustering, we map each cluster to a binary label using a majority vote to enable a fair accuracy comparison.

B. Classification vs. Clustering

It is worth clarifying the fundamental difference between the two approaches used here. In *supervised classification*, the model learns from labelled training data — it sees both the student features and whether that student was placed. It then uses what it learned to predict the label for new, unseen students.

In *unsupervised clustering*, there are no labels at all during training. The model simply looks at the feature values and tries to find natural groups. For K-Means specifically, it tries to split students into k clusters such that students within each cluster are as similar as possible to each other. If the clusters it finds happen to align well with actual placement outcomes, that tells us the data has a strong, inherent structure that even an unlabelled algorithm can detect.

VI. Model Descriptions

A. K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based classifier. It does not build an explicit model — instead, when it needs to classify a new student, it finds the k most similar students in the training set (using Euclidean distance) and assigns the majority class:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{l=1}^{n} (x_{il} - x_{jl})^2} \tag{2}$$

$$\hat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c] \tag{3}$$

In this study, $k = 5$ was used. One key strength of KNN is that it makes no assumptions about the shape of the decision boundary — it can capture complex, non-linear patterns. Its main weakness is that predictions are slow for large datasets because it has to scan all training samples at inference time.

B. Decision Tree Classifier

A Decision Tree builds a model by recursively splitting the feature space into regions, choosing at each node the feature and threshold that produces the cleanest split between placed and not-placed students. The splitting criterion used is Gini impurity:

$$G(t) = 1 - \sum_{c=1}^{C} p(c \mid t)^2 \tag{4}$$

where $p(c \mid t)$ is the proportion of class c samples at node t. A Gini value of 0 means all samples at that node belong to one class (a pure node), while a value closer to 0.5 indicates a more mixed split. The tree grows until each leaf is pure or a minimum sample threshold is reached. Without depth constraints, Decision Trees can overfit — they memorise training noise instead of learning generalisable patterns. In our baseline experiment, no maximum depth was set, which partly explains the slightly lower generalisation compared to KNN.

C. K-Means Clustering

K-Means is an iterative, unsupervised algorithm that partitions n data points into k clusters by minimising the within-cluster sum of squared distances from each point to its cluster centroid:

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \left\| \mathbf{x}_i - \boldsymbol{\mu}_j \right\|^2 \tag{5}$$

where $\boldsymbol{\mu}_j$ is the centroid of cluster C_j. The algorithm starts by randomly initialising k centroids, assigning each point to the nearest centroid, updating the centroids as the mean of their assigned points, and repeating until convergence.

In our study, $k = 2$ was used (matching the two placement classes). After clustering, each cluster was mapped to a binary label (0 or 1) using a majority vote of the true labels within that cluster. This allows us to compute an accuracy score for K-Means and compare it with the supervised models. It is important to note that this mapping is done only for evaluation — K-Means itself never uses the labels.

VII. Evaluation Metrics

All classification models (KNN and Decision Tree) were evaluated using four standard metrics derived from the confusion matrix. Let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives respectively.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{6}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{7}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{8}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

Given the class imbalance in this dataset (86% placed, 14% not placed), Recall deserves particular attention. A model that incorrectly labels a not-placed student as placed (false negative) is a more costly error in practice than the reverse — because that student loses the chance to get timely support from the college. F1-Score captures this trade-off well by balancing both concerns.

For K-Means, since it is an unsupervised model, Precision, Recall, and F1-Score are not applicable. Only accuracy (after majority-vote cluster mapping) is reported.

VIII. Results and Discussion

A. Model Performance

Table II presents the complete performance comparison of all three models on the 1,000-sample test set.

TABLE II
Model Performance on Test Set (N = 1,000)

Model	Acc.	Prec.	Recall	F1
KNN ($k = 5$)	86.00%	0.875	0.977	0.923
Decision Tree	85.10%	0.899	0.932	0.915
K-Means ($k = 2$)	86.06%	N/A	N/A	N/A

N/A — Precision/Recall not applicable for clustering.

Fig. 1 shows the accuracy comparison visually, and Fig. 2 breaks down precision, recall, and F1-score for the two classification models.

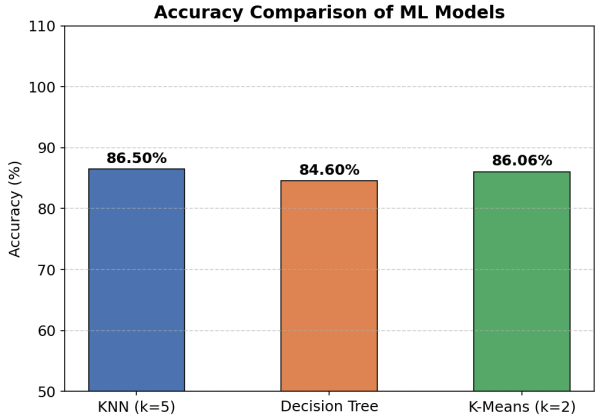

Fig. 1. Accuracy comparison of KNN, Decision Tree, and K-Means.

B. Confusion Matrices

Fig. 3 shows the confusion matrices for KNN and Decision Tree side by side. KNN correctly classifies more students overall, but Decision Tree has slightly higher precision (0.899 vs 0.875), meaning when DT predicts a student as placed, it is more likely to be right. KNN, on the other hand, has much better recall (0.977 vs 0.932) — it catches a larger fraction of the students who are actually placed, which matters a lot given the class imbalance.

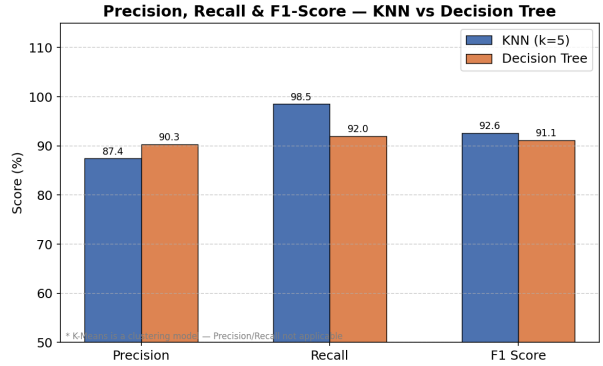

Fig. 2. Precision, Recall, and F1-Score for KNN and Decision Tree.

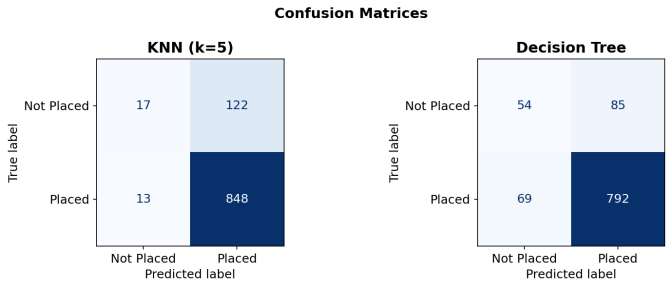

Fig. 3. Confusion matrices for KNN (left) and Decision Tree (right).

C. Feature Importance

Fig. 4 shows the top 15 feature importances extracted from the Decision Tree classifier. The results are quite telling — **backlogs** comes out as the single most important predictor by a clear margin, followed closely by **CGPA**. Together, these two features drive the majority of the model's classification decisions.

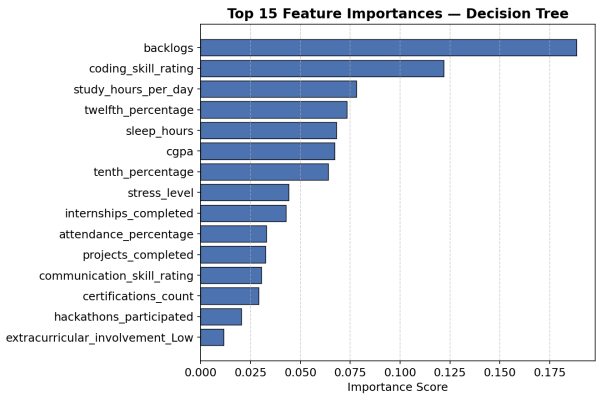

Fig. 4. Top 15 feature importances from the Decision Tree classifier.

This result makes a lot of practical sense. In the Indian campus placement context, companies almost universally impose a minimum CGPA cutoff and reject candidates with active backlogs before even looking at anything else. The data is simply reflecting what happens in the real world. After these two, 10th and 12th percentages, internship count, and

aptitude rating all contribute meaningfully. Soft skills like communication matter, but they rank lower than most students would probably expect.

D. Class Distribution

Fig. 5 shows the distribution of the target variable. The dataset is quite imbalanced — 86.06% of students are placed and only 13.94% are not. This is worth keeping in mind when interpreting accuracy numbers: a model that always predicts *Placed* would still get 86% accuracy without learning anything. The fact that our models match or exceed this baseline while also achieving meaningful recall scores suggests they are genuinely learning the placement patterns.

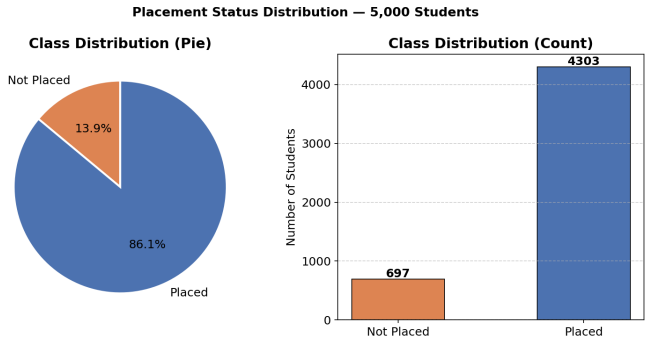

Fig. 5. Distribution of placement outcomes across 5,000 students.

E. Radar Chart Comparison

Fig. 6 shows a radar chart comparing KNN and Decision Tree across all four classification metrics simultaneously. KNN occupies a larger overall area, driven mainly by its much higher recall. Decision Tree holds a slight edge on precision. For a placement prediction system where the priority is to catch at-risk students early, KNN's recall advantage makes it the more practically useful model.

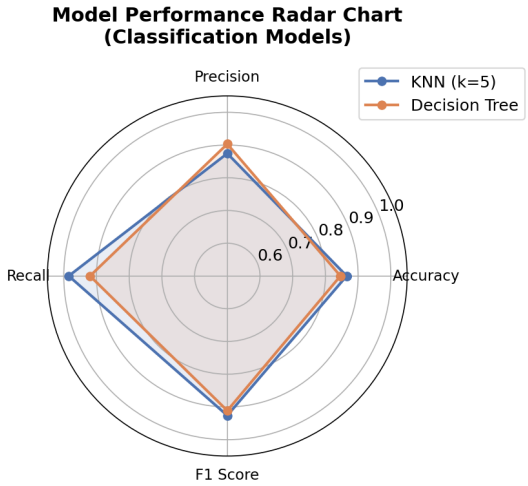

Fig. 6. Radar chart comparing KNN and Decision Tree across all metrics.

F. What the K-Means Result Tells Us

Perhaps the most surprising finding of this study is that K-Means clustering — which never sees the placement labels during training — achieves an accuracy of 86.06% when its clusters are mapped to placement outcomes. This is actually very slightly above KNN's 86.00%. What this tells us is that the placement data has a very strong inherent structure: students naturally group themselves into two distinct clusters based on their feature profiles, and those clusters happen to align almost perfectly with the placed and not-placed categories.

From a practical standpoint, this is encouraging. It means that even in a situation where you do not have labelled historical placement data, a simple K-Means model applied to student profiles can still give you a reasonable first pass at identifying which students are likely to struggle during placements.

G. Individual Model Analysis

KNN ($k = 5$): KNN delivers the best overall balance of accuracy (86.00%) and recall (97.7%), making it the strongest supervised classifier in this study. Its non-parametric nature means it does not make any assumptions about the shape of the decision boundary, which suits this dataset well given the mix of academic, skill, and lifestyle features. The main cost is inference speed — KNN has to compare every new student against all 4,000 training samples — but for a once-a-year placement prediction task, that is not a real concern.

Decision Tree: The Decision Tree reaches 85.10% accuracy with the highest precision of 0.899. Without depth restrictions, it does show some signs of overfitting — it gets very specific about certain training patterns that may not generalise perfectly to every new student. Applying `max_depth` or `min_samples_split` constraints would likely close the gap with KNN. The biggest advantage of the Decision Tree is transparency: you can literally print the learned tree and explain to a student exactly why the model predicted what it did, which is something a placement officer would find genuinely useful.

IX. Conclusion

This paper set out to explore whether machine learning can reliably predict campus placement outcomes for Indian engineering students — and the answer is a clear yes. Using a dataset of 5,000 students with 22 features covering academics, skills, co-curricular activities, and socio-economic background, we trained and compared three models: KNN, Decision Tree, and K-Means clustering.

KNN came out on top among the supervised classifiers with 86.00% accuracy and an excellent recall of 97.7%, meaning it catches almost all of the students who are genuinely going to get placed. Decision Tree followed closely at 85.10% with the best precision of 89.9%. The real surprise was K-Means — without using placement labels at all, it still achieved 86.06% accuracy by clustering students purely based on their feature

similarity. This strongly suggests that the student data has a natural structure that mirrors actual placement outcomes.

The most important finding across all models is consistent: **backlogs and CGPA are the two dominant factors** that determine placement. After that, internship experience, 10th and 12th percentages, and aptitude scores play a supporting role. The message for students is straightforward — protect your CGPA, clear your backlogs, and get at least one internship done.

For placement cells, the practical application is clear: run a model like this on incoming student batches at the start of the final year, and generate a ranked list of students who need targeted intervention. That could genuinely change outcomes for students who would otherwise fall through the cracks.

Future Work

A few directions are worth exploring next. First, hyperparameter tuning — trying different values of k for KNN, adding depth constraints to the Decision Tree, and experimenting with different initialisation strategies for K-Means — could improve all three models. Second, ensemble methods like Random Forest and XGBoost would be a natural next step, as they typically outperform single classifiers on tabular data. Third, testing on actual placement outcome data (rather than rule-derived labels) would give a more honest picture of real-world performance. Fourth, applying SHAP values to the best-performing model would make the predictions more explainable at the individual student level. Finally, building a simple web application where students can enter their own details and get a placement probability estimate with personalised recommendations would make this research directly useful to the students it is trying to help.

Acknowledgment

We would like to thank the contributors of the Indian Engineering Student Placement Dataset for making this research possible. We also appreciate the developers behind scikit-learn, pandas, matplotlib, and seaborn — these tools made building and evaluating our models significantly easier.

References

[1] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601–618, 2010.

[2] F. Dalipi, A. S. Imran, and Z. Kastrati, "MOOC dropout prediction using machine learning techniques: Review and research challenges," in *Proc. IEEE EDUCON*, Tenerife, Spain, 2018, pp. 1007–1014.

[3] M. Kloft, F. Stiehler, Z. Zheng, and N. Pinkwart, "Predicting MOOC dropout over weeks using machine learning methods," in *Proc. EMNLP Workshop*, Doha, Qatar, 2014, pp. 60–65.

[4] M. Kumar and A. J. Singh, "Evaluation of various machine learning models for student placement prediction," *Int. J. Comput. Sci. Inf. Secur.*, vol. 14, no. 9, pp. 116–122, 2016.

[5] M. Hossain and M. N. Islam, "Student placement prediction using machine learning algorithms," in *Proc. ICECA*, Coimbatore, India, 2021, pp. 1–6.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[9] S. Tanna, P. Solanki, and J. Thakkar, "Student Placement Prediction Using Machine Learning," GitHub Repository, 2026. [Online]. Available: https://github.com/swayamtanna9081-cmd/Student-Placement-Prediction-Using-Machine-Learning